L3

Physical Storage and Data Organization



Key Problem

How does a logical table become physical bytes that the DBMS can read, write, cache, and measure?

Logical schema → Record layout → Page / block → File / tablespace → Buffer pool → **Measured footprint**

Why this matters

- Storage choices shape I/O cost, memory behavior, and project feasibility.
- A database with the same logical rows can have different physical footprints across DBMSs.

What you will do

- Calculate a first-order storage estimate,
- load data into MySQL, measure DATA_LENGTH / INDEX_LENGTH, and explain the gap.

Project artifact

Each group will produce a Storage & Scale Profile:

- expected scale,
- measured footprint,
- growth projection,
- DBMS comparison.

Part A

From Records to Pages

Calculate how logical rows occupy physical blocks/pages before looking at MySQL.

Storage Hierarchy and Volatility

Fast storage is expensive; persistent storage is slower but keeps data after failures.

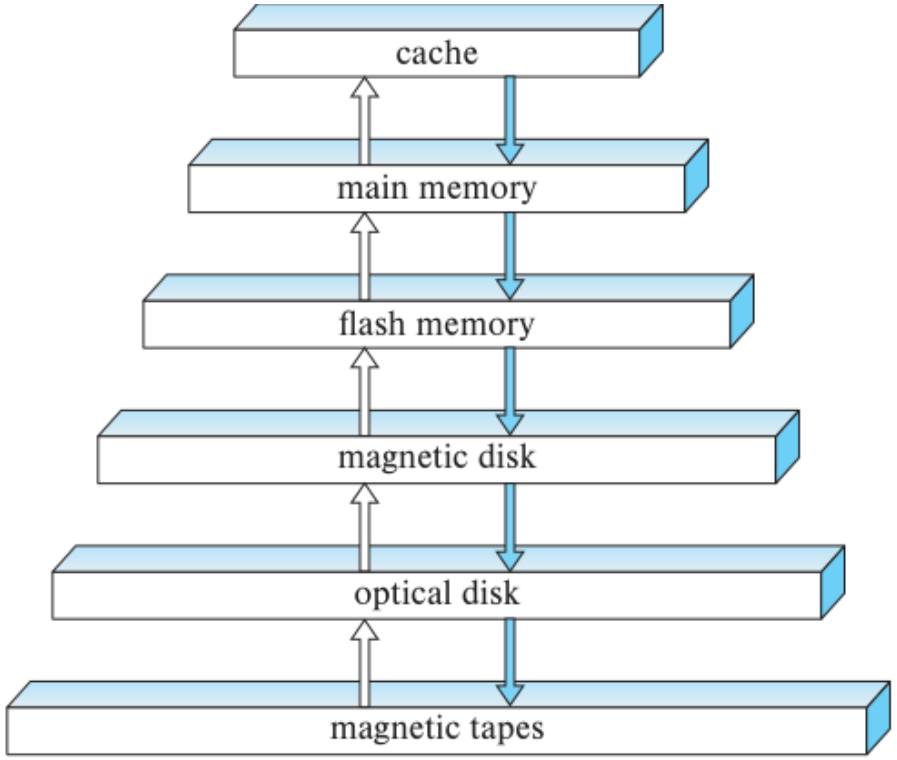


Figure 12.1 Storage device hierarchy.

Core interpretation

Higher levels are faster but smaller and more expensive. Lower levels are slower but cheaper and persistent.

DBMS implication

A DBMS must decide which pages stay in memory and which pages live only on persistent storage.

Source: DSC 7e Ch.12, Fig. 12.1 Storage device hierarchy.

Page / Block as the Unit of Transfer

A file is logically partitioned into fixed-size blocks; records are placed inside blocks.

Definitions

- File: persistent storage object managed by the DBMS/OS
- Block/page: fixed-length storage and transfer unit
- Record: physical representation of one tuple / row

Typical sizes

- DSC examples: 4–8 KB blocks
- PostgreSQL default page: 8 KB
- InnoDB default page: 16 KB

Important

- The DBMS usually pays I/O at page granularity.
- A row read can cause a full page read.

Fixed-Length Record Example

Start with a simple tuple whose fields have fixed sizes.

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Example tuple

instructor(ID, name, dept_name, salary)

Attribute	Size
ID	5 bytes
name	20 bytes
dept_name	20 bytes
salary	8 bytes

Formula 1 - Record Size

For a simple fixed-length record, add the field sizes.

$$R = \Sigma\ \text{size}(A_i)$$

R = record size in bytes

Worked calculation

ID 5 + name 20 + dept_name 20 + salary 8 = **53** bytes

$$R = 5 + 20 + 20 + 8 = \mathbf{53}\ \text{bytes}$$

This is a conceptual record size, not the exact InnoDB row size.

Formula 2 - Blocking Factor

How many records fit completely inside one block/page?

$$bfr = \lfloor B / R \rfloor$$

B = block/page size; R = record size

$$bfr = \lfloor 4096 / 53 \rfloor = 77 \text{ records/page}$$

Use floor: a partial record does not count.

Why floor?

A record that crosses a block boundary may require two block accesses.

$$Unused = B - bfr \times R$$

Wasted bytes per page in the simple fixed-record model.

$$\textbf{Unused} = 4096 - 77 \times 53 = 15 \text{ bytes/page}$$

Even before DBMS overhead, physical packing can leave unused space.

Formula 3 - Number of Pages and Storage

Convert a row count into page count and storage footprint.

$$b = \lceil N / bfr \rceil$$

N = number of records;
b = number of blocks/pages

$$S = b \times B$$

S = estimated physical storage for data pages

$$b = \lceil 10{,}000 / 77 \rceil = 130 \text{ pages}$$

Example with 10,000 instructor records

$$S = 130 \times 4096 = 532{,}480 \text{ bytes} \approx 520 \text{ KiB}$$

This is before DBMS row/page metadata and indexes.

In-Class Exercise — Change the Page Size

Students should practice the same chain with a different assumption.

Given

Record size R = 128 bytes

Page size B = 16 KiB = 16,384 bytes

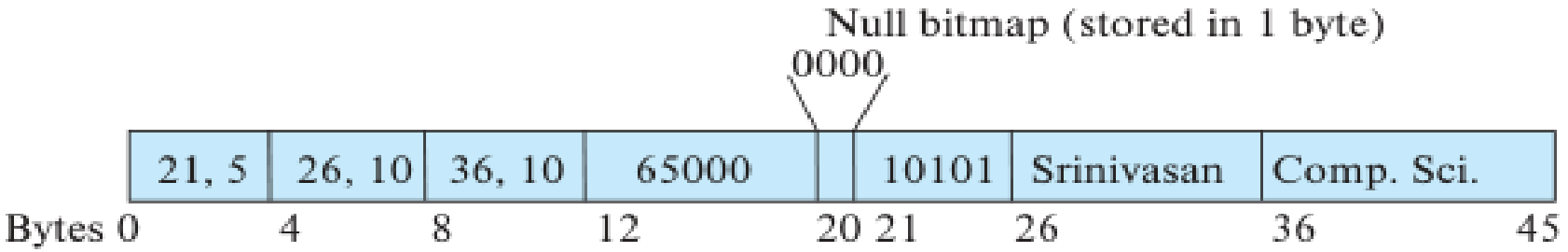
Number of records N = 1,000,000

Compute

1. bfr = ?
2. number of pages b = ?
3. estimated data storage S = ?
4. Why is this only an estimate for InnoDB?

Variable-Length Records: Estimate, Not Exact Formula

Real rows often contain VARCHAR, NULLs, and metadata.



$$R_{avg} = R_{fixed} + R_{variable,avg} + R_{metadata}$$

Use average payload length and state assumptions.

What changes?

- VARCHAR length varies across rows
- NULL bitmap may be stored
- Offsets/lengths can be needed
- DBMS row format adds metadata
- Updates can create free space/fragmentation

Formula 4 - Approximate Capacity for Variable Rows

Use average row size and usable page bytes.

$$bfr_{approx} \approx \lfloor PageUsableBytes / (R_{avg} + SlotOverhead) \rfloor$$

Conceptual estimate: page header, slot directory, row metadata, and free space reduce capacity.

Example assumptione

Page = 16,384 B
Usable = 15,500 B
R_{avg} = 220 B
Slot overhead ≈ 4 B

$$bfr_{approx} \approx \lfloor 15{,}500 / (220 + 4) \rfloor = 69 \text{ rows/page}$$

This helps predict order of magnitude before measuring.

Slotted Page: Why Variable-Length Records Still Work

A slotted page uses indirection: slot entries point to record locations.

Header stores #slots → Slots store location+size → Records grow from opposite side → **Free space stays in middle**

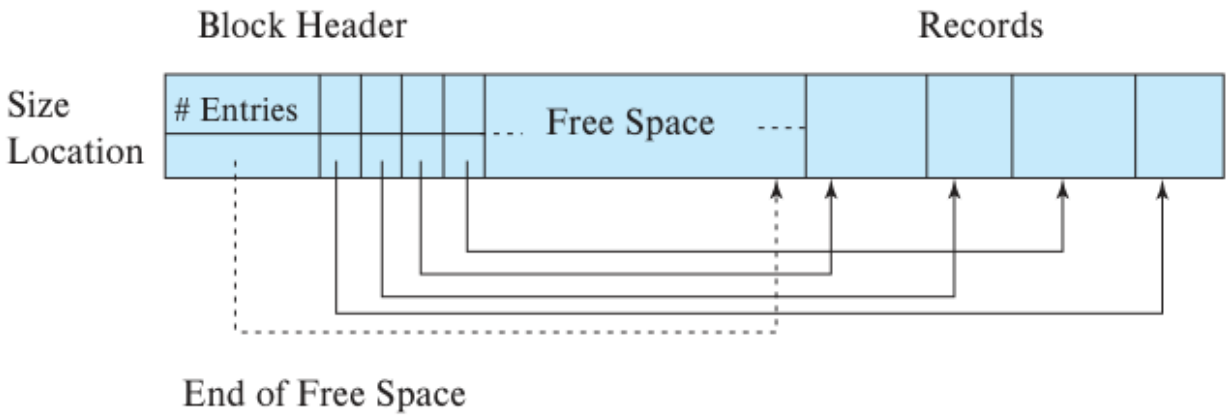


Figure 13.6 Slotted-page structure.

Insert

Add a slot entry and place the record in free space. If space is insufficient, choose another page or reorganize.

Delete / resize

Mark slot deleted or update slot metadata; records can move within the page while slot identity remains stable.

Estimation Error

Students must quantify the gap, not just say “there is overhead.”

$$\text{Error}(\%) = ((\text{Measured} - \text{Estimated}) \,/\, \text{Estimated}) \times 100$$

Use the same unit for Measured and Estimated.

Example

Estimated = 240 MiB
Measured = 310 MiB
Error = (310 − 240) / 240 × 100 = 29.2%

Explain possible causes

- row/page metadata
- page free space
- variable-length values
- clustered primary key
- secondary indexes
- fragmentation / data_free

Project reports should include both the number and a reasoned explanation.

Part B

File Organization and Storage Models

Physical layout should be chosen for the access pattern, not only for elegance.

Heap File: Simple, Flexible, Unordered

Good for inserts; retrieval needs scan or an access path such as an index.

Main idea

A heap file stores records where space is available. It is not sorted by a search key. Inserts are easy; finding matching rows requires scanning unless an index exists.

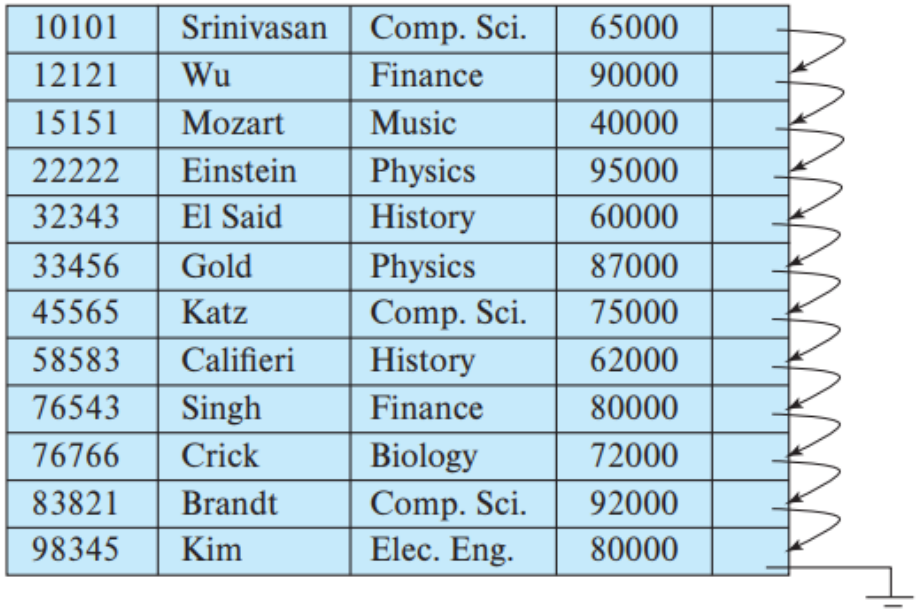


Operation	Heap behavior
Insert	Find a page with enough free space
Full scan	Read many/all pages
Point lookup	Needs an index or scan
Delete	Free space must be tracked

Use heap-file reasoning when the access pattern is write-heavy or not naturally sorted.

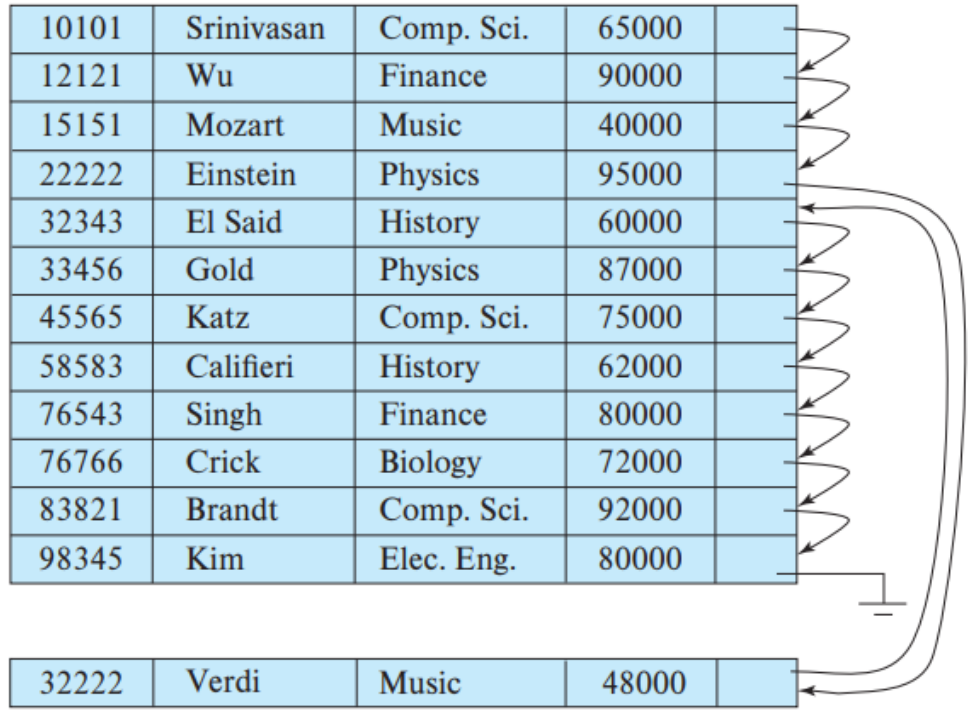
Sequential File: Fast Ordered Processing, Costly Maintenance

Records are stored close to search-key order.



10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	

Figure 13.7 Sequential file for *instructor* records.



10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	
32222	Verdi	Music	48000	

Figure 13.8 Sequential file after an insertion.

- Queries that scan data in search-key order or benefit from physically sorted input.
- Insertions can create overflow blocks or break physical order over time.
- Reorganization may restore locality but has operational cost and should be scheduled carefully.

Multitable Clustering: Store Related Records Together

Physical locality can reduce block reads for common joins.

Comp. Sci.	Taylor	100000	
10101	Srinivasan	Comp. Sci.	65000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000
Physics	Watson	70000	
33456	Gold	Physics	87000

Figure 13.11 Multitable clustering file structure.

Query pattern

department ⋈ instructor is common, so storing each department tuple near its instructor tuples improves locality.

Trade-off

Helps one join pattern, but may complicate updates and other access patterns.

Project lesson

Physical layout is a workload decision, not only a schema decision.

File Organization Decision Exercise

Choose layout using workload evidence.

Workload pattern	Preferred idea	Reason
Frequent inserts, many ad-hoc filters	Heap + indexes	insert flexibility, access paths for lookup
Frequent sorted scan by key	Sequential / clustered access	physical order reduces random I/O
Frequent join on related entities	Clustering / denormalization candidate	related records read together
Frequent aggregate over few columns	Columnar system candidate	avoid reading unused columns

In project reports, do not write “DBMS A is faster.”
Write: “Under workload W and scale N, this layout/access path reduced the pages that had to be read.”

Row-Oriented vs Column-Oriented Storage

The same relation can be physically organized very differently.

10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Figure 13.14 Columnar representation of the *instructor* relation.

Row-oriented

Attributes of one tuple are stored together. Good for OLTP-style access that needs full rows.

Column-oriented

Values of one attribute are stored together. Good for analytical scans over few columns and many rows.

Project direction

MySQL/PostgreSQL vs ClickHouse is a strong comparison for aggregate workloads.

Source: DSC 7e Ch.13 §13.6, Fig. 13.14 Columnar representation.

Workload Exercise: What Should Be Read?

Use this to motivate modern analytical DBMSs.

```
SELECT AVG(amount)
FROM sales
WHERE year = 2026;
```

Assume sales has 30 columns

The query needs only amount and year. A row-store may bring many irrelevant attributes with each row; a column-store can read fewer columns.

Question	Answer to discuss
Which columns are required?	year and amount
Does each query need full rows?	No
Which layout may reduce I/O?	Column-oriented
What is the trade-off?	Updates and point-row access can be less natural

Part C

Buffer Pool and InnoDB Mapping

Connect the page model to a running MySQL system.

Why a Buffer Manager Exists

The database can be much larger than memory, but queries need pages in memory.

Operator requests page P → Buffer lookup → Hit: return frame → Miss: read from storage → **Continue query**

Concept

- The buffer is the part of main memory used for copies of disk blocks.
- The buffer manager allocates buffer space, fetches missing blocks, and writes back dirty blocks when needed.

Practical

- If a page is already in memory, query execution avoids a physical read.
- If not, a storage read is required before the operator can proceed.

Formula 5 - Buffer Hit Ratio

Measure the difference between logical page requests and physical reads.

$$\text{HitRatio} = 1 - (\text{PhysicalReads} \,/\, \text{LogicalReadRequests})$$

Use deltas before/after a workload, not only absolute counters.

MySQL counters

ΔInnodb_buffer_pool_read_requests = logical read requests
ΔInnodb_buffer_pool_reads = reads that had to go to disk

$$\text{HitRatio} = 1 - 2{,}000 \,/\, 80{,}000 = 97.5\%$$

Caution: OS cache and workload warm-up can affect timing; counters are better than stopwatch-only evidence.

InnoDB Mapping: From Table to Pages

In this lecture, we use InnoDB as the running DBMS example.

Table → Clustered index data pages → Tablespace file(s) → InnoDB buffer pool → **Query operators**

Page size

Check with:
SHOW VARIABLES LIKE 'innodb_page_size';
Default is commonly 16 KB.

Data length

For InnoDB, DATA_LENGTH is approx clustered index size in pages × page size.

Index length

INDEX_LENGTH is approx non-clustered index space in pages × page size.

```
SHOW VARIABLES LIKE 'innodb_page_size';
SHOW VARIABLES LIKE 'innodb_buffer_pool_size';
```

Measuring Table Footprint in MySQL

Use metadata as evidence, not only screenshots of query output.

```
SELECT table_name, table_rows, avg_row_length,
       data_length, index_length, data_free
FROM information_schema.tables
WHERE table_schema = 'l3_demo'
ORDER BY data_length + index_length DESC;
```

Interpretation

TABLE_ROWS can be approximate for InnoDB. Use SELECT COUNT(*) when you need an exact row count in the lab.

Column	Use in L3 lab
DATA_LENGTH	approx allocated clustered-index data bytes
INDEX_LENGTH	approx allocated secondary-index bytes
DATA_FREE	allocated but unused space
AVG_ROW_LENGTH	coarse average row length

Formula 6 - Convert Measured Bytes to Pages

After measuring MySQL, translate bytes back into physical units.

$$\text{DataPages_approx} = \text{DATA_LENGTH} / \text{innodb_page_size}$$

Approximate pages allocated to clustered data.

$$\text{IndexPages_approx} = \text{INDEX_LENGTH} / \text{innodb_page_size}$$

Approximate pages allocated to secondary indexes.

Example

DATA_LENGTH = 83,886,080 bytes
page_size = 16,384 bytes

$$83{,}886{,}080 / 16{,}384 = 5120 \text{ pages}$$

This gives students a physical interpretation of metadata.

Formula 7 — Physical Bytes Per Row

This is the project-friendly metric students can extrapolate.

$$\text{PhysicalBytesPerRow} = (\text{DATA_LENGTH} + \text{INDEX_LENGTH}) \,/\, \text{COUNT}(*)$$

Use exact COUNT(*) for the denominator in the lab.

Example

COUNT(*) = 100,000
DATA + INDEX = 32 MiB
PhysicalBytesPerRow ≈ 335.5 B/row

$$\text{ProjectedSize}(N_t) = \text{PhysicalBytesPerRow} \times N_t$$

First-order projection to target scale.

Add a warning in reports: projection assumes similar row distribution, indexes, storage configuration, and workload evolution.

What Counts as Good Measurement Evidence?

The report should allow someone else to reproduce and interpret the result.

Evidence	Required output
Schema	CREATE TABLE / DBMS version / engine
Scale	COUNT(*) at each scale point
Estimate	R_avg, bfr or estimated bytes
Measurement	DATA_LENGTH, INDEX_LENGTH, DATA_FREE
Calculation	pages, bytes/row, error %, projection
Explanation	why estimate and measurement differ

Part D

Lab

Predict first. Measure second. Explain the difference.

Live Demo Storyline

A small MySQL experiment

Create tables → Predict storage → Load 100K+ rows → Measure footprint → Add index → Remeasure → **Explain**

Table A: narrow

id BIGINT
code CHAR(10)
created_at DATETIME
small payload

Table B: wide

same key
VARCHAR(500) description
variable payload length

What to observe

Same row count can produce very different data pages, index pages, and bytes/row.

Demo Step 1 — Create Tables

Keep the schema simple enough that students can reason about row width.

```
CREATE DATABASE IF NOT EXISTS
l3_demo;
USE l3_demo;

CREATE TABLE storage_narrow (
  id BIGINT PRIMARY KEY,
  code CHAR(10),
  created_at DATETIME
) ENGINE=InnoDB;
```

```
CREATE TABLE storage_wide (
  id BIGINT PRIMARY KEY,
  code CHAR(10),
  description VARCHAR(500),
  created_at DATETIME
) ENGINE=InnoDB;
```

Before loading rows, ask: which table should use more DATA_LENGTH, and why?

Demo Step 2 - Generate Data at Multiple Scales

Use enough rows to make storage effects visible.

```
-- Example scale points
-- 10K  → quick sanity check
-- 100K → classroom demo
-- 500K / 1M → homework or project

CALL load_storage_demo(100000);
```

- The exact generator can be a stored procedure, Python script, or application script.
- What matters is repeatability and controlled scale.

Scale	Purpose
10K	check script and schema
100K	classroom measurement
500K+	project / homework
1M+	stronger evidence if machine allows

Demo Step 3 - Estimate Before Measuring

Table	R_avg assumption	N	Estimated bytes
storage_narrow	~ 40–60 B/row	100K	~ 4–6 MB
storage_wide	~ 220–400 B/row	100K	~ 22–40 MB

$$\text{EstimatedLogicalSize} = N \times R_avg$$

This is a logical estimate before page/index overhead.

$$\text{Pages_est} = \lceil \text{EstimatedLogicalSize} / \text{PageUsableBytes} \rceil$$

Optional if students estimate page count.

Demo Step 4 - Measure Storage in MySQL

Now compare prediction with observed metadata.

```
ANALYZE TABLE storage_narrow, storage_wide;

SELECT table_name, table_rows, data_length,
       index_length, data_free, avg_row_length
FROM information_schema.tables
WHERE table_schema = 'l3_demo'
  AND table_name LIKE 'storage_%';
```

What to record

- exact COUNT(*)
- DATA_LENGTH
- INDEX_LENGTH
- DATA_FREE
- page size
- DBMS version

Observation	Interpretation
wide > narrow	larger variable payload
index_length grows after CREATE INDEX	secondary index pages added
data_free > 0	allocated but unused space exists

Demo Step 5 — Add an Index and Remeasure

Index storage is physical storage too.

```
CREATE INDEX idx_wide_code
ON storage_wide(code);

ANALYZE TABLE storage_wide;

SELECT data_length, index_length
FROM information_schema.tables
WHERE table_schema = 'l3_demo'
  AND table_name = 'storage_wide';
```

Expected result

DATA_LENGTH may stay similar, while INDEX_LENGTH should increase because the secondary index requires its own pages.

$$\text{TotalFootprint} = \text{DATA_LENGTH} + \text{INDEX_LENGTH}$$

Use total footprint for project storage planning.

This prepares Lecture 4: indexes speed access but add storage and write-maintenance cost.

Demo Step 6 - Compute Pages, Bytes/Row, Error

Do not stop at raw metadata; transform it into evidence.

$$\text{ObservedPages} \approx \text{DATA_LENGTH} / \text{page_size}$$

Physical interpretation of data bytes.

$$\text{PhysicalBytesPerRow} = (\text{DATA_LENGTH} + \text{INDEX_LENGTH}) / \text{COUNT}(*)$$

Project-friendly metric.

$$\text{Error}(\%) = ((\text{Measured} - \text{Estimated}) / \text{Estimated}) \times 100$$

Quantify the difference.

$$\text{ProjectedStorage}(N_t) = \text{PhysicalBytesPerRow} \times N_t$$

Extrapolate to target workload.

Required reasoning

If measured storage differs from estimate, explain using row metadata, page allocation, indexes, free space, and variable-length data distribution.

Demo Step 7 - Buffer Pool Counter Experiment

Use counters to show memory vs storage behavior.

```
SHOW GLOBAL STATUS LIKE 'Innodb_buffer_pool_read%';

-- Run a scan or selective query
SELECT COUNT(*) FROM storage_wide
WHERE code LIKE 'A%';

SHOW GLOBAL STATUS LIKE 'Innodb_buffer_pool_read%';
```

Measure deltas

- Δ read_requests = logical reads
- Δ reads = physical reads
- Hit ratio = 1 − reads / read_requests
- Repeat query and compare

Do not overpromise timing.
Use counters and explanation; wall-clock time depends on OS cache and machine state.

Comparing Two DBMSs: Same Problem, Appropriate Models

Do not force identical physical schemas across different DBMSs.

Same application problem → DBMS A model → DBMS B model → Same logical workload → **Compare evidence**

Dimension	Relational example	Document/search/columnar example
Logical object	orders, order_items	orders document / index document / columnar table
Measured footprint	DATA_LENGTH + INDEX_LENGTH	collection size + index size / segment size
Explanation	normalization, indexes	embedding, analyzer index, compression
Conclusion	workload-specific	not absolute winner

Good conclusion format:
“Under workload W, scale N, configuration C, system A used X while system B used Y, likely because …”

Part E

Beyond the Core

Use these resources for stronger projects and independent exploration.

Advanced Exploration 1 — Real Page Layouts

PostgreSQL documents page structure very explicitly.

Explore further

PostgreSQL stores tables and indexes as arrays of fixed-size pages, usually 8 KB. Its page layout documentation describes page header, item identifiers, free space, and tuple area.

Project idea

Compare how PostgreSQL and MySQL expose storage metadata. Which system makes page-level reasoning easier for a student project?

Core takeaway: real DBMS page layouts instantiate the same page/record ideas from the textbook.

Source: PostgreSQL 18 Documentation: 66.6 Database Page Layout.

Advanced Exploration 2 — Buffer Pool Tuning

MySQL goes much deeper than our L3 core.

Explore further

InnoDB buffer pool uses an LRU-like list with young and old sublists, read-ahead, flushing, and scan-resistance mechanisms.

Project idea

For a read-heavy workload, measure buffer hit behavior before/after changing workload order or memory size. Do not change production settings blindly.

Core takeaway: buffer behavior is measurable and tunable, but deeper tuning belongs to advanced project work.

Source: MySQL 8.4 Reference Manual §17.5.1 Buffer Pool; MIT 6.830 Lecture 7.

Advanced Exploration 3 — Columnar Systems and File Formats

Column-oriented ideas lead directly to modern analytical systems.

ClickHouse

A columnar DBMS for analytical queries. Strong project direction for GROUP BY / aggregate workloads at scale.

Apache Parquet

A columnar file format organized by row groups, column chunks, and pages. Useful for data-lake style projects.

Project comparison

MySQL/PostgreSQL vs ClickHouse or Parquet-based analytics: storage footprint, compression, and query scan cost.

Core takeaway: storage model and workload must match; analytical storage is not just a faster version of OLTP storage.

Source: DSC 7e §13.6; ClickHouse official docs; Apache Parquet file format documentation.

Advanced Exploration 4 — Implementation-Oriented Courses

For students who want to understand DBMS internals more deeply.

Course/source	Relevant unit	Why useful
CMU 15-445/645	Database Storage I / Memory Management	pages, buffer pool, storage manager, implementation projects
Berkeley CS186	Disks, Buffers, Files	clear teaching sequence before indexing
MIT 6.830	Buffer pool design and memory management	why DBMS buffer management differs from OS VM
MySQL Manual	InnoDB internals and metadata	official behavior for this course lab

These are recommended for project extension, not mandatory exam material unless explicitly assigned.

Further Reading and Resources

Use these to extend the project, not to replace the core calculation method.

Purpose	Recommended resource
Core reading	Database System Concepts 7e, Ch.12–13
MySQL lab reference	MySQL 8.4 Manual: INFORMATION_SCHEMA.TABLES, InnoDB Buffer Pool, InnoDB INFORMATION_SCHEMA
Page internals	PostgreSQL docs: Database Page Layout
Implementation perspective	CMU 15-445/645 Storage and Memory Management
Teaching notes	Berkeley CS186 Disks, Buffers, Files; MIT 6.830 Lecture 7
Analytical storage	DSC §13.6; ClickHouse; Apache Parquet file format

When citing external material in a project, cite official documentation or course notes whenever possible.